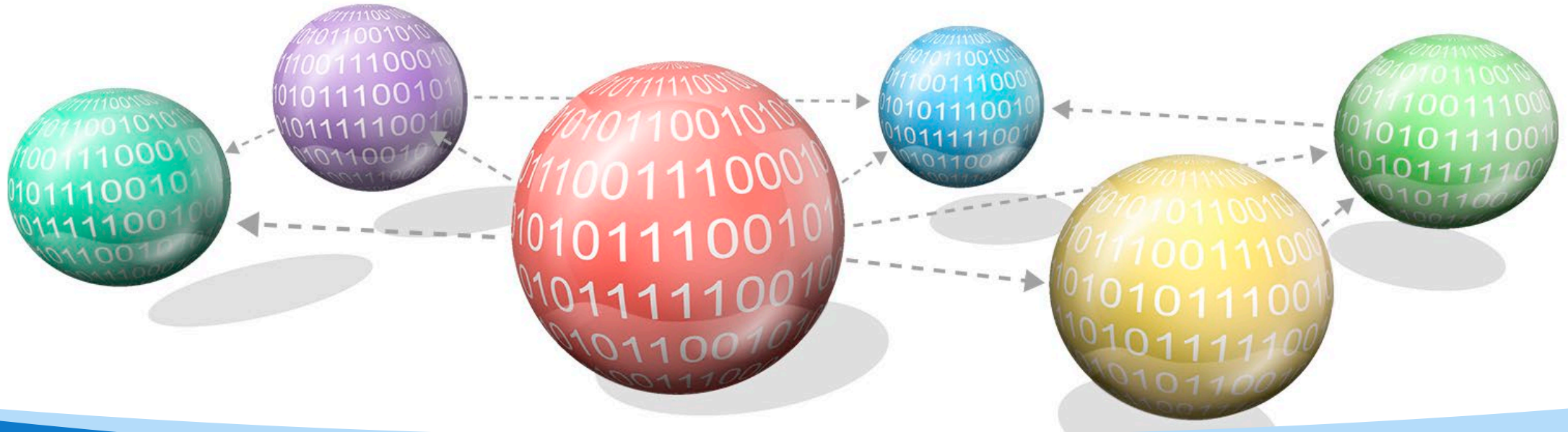




SC2002 Object-Oriented Design & Programming

Chapter 7: UML Model: Object Interactions - Sequence Diagram

Dr. Li Fang
Senior Lecturer, CCDS



**NANYANG
TECHNOLOGICAL
UNIVERSITY**
SINGAPORE

sequence diagram

- Sequence diagrams, commonly used by developers, model the interactions between objects in a single **use case**. They illustrate how the different parts of a system interact with each other to carry out **a function**, and the order in which the interactions occur when a particular use case is executed.
- In simpler words, a sequence diagram shows different parts of a system work in a 'sequence' to get something done.



What does a Use Case describe?

Definition of a Use Case

- “A Use Case specifies a set of actions performed by a system, which yields an observable result of value to a particular actor.”
— *Unified Modeling Language (UML 2.x Specification)*

Key Characteristics

Element	Description
Actor	The external user or system that initiates the interaction.
System	The software or process being described.
Goal	The result or outcome the actor wants to achieve.
Main Success Scenario	The normal flow of events that lead to the goal.
Alternate / Exception Flows	Variations or error conditions in the process.

----- Online shopping steps -----



1. Select items



2. Add to cart



3. Order preparation



4. Delivery to your door



5. Receiving an order

Example of Atomic / Low-level use case

A member places an order containing one or more order items. The order system then processes the items one by one. For each order item, the system checks the shipping method selected for that item. If the shipping method is courier delivery, the order system sends a dispatch request to the courier service to deliver the item. If the shipping method is mail delivery, the order system sends a dispatch request to the mail service to deliver the item. This process is repeated for every item in the order. After all items have been dispatched, the system checks whether a confirmation is required. If confirmation is needed, the order system sends a confirmation message through the notification service.

CD vs SD

- Think of the **Class Diagram** as the **blueprint of the entire online shopping system**, showing all the classes such as Customer, Order, OrderItem, Product, Payment, ShippingService, and Notification, and how they are related to each other.
- The **Sequence Diagram**, on the other hand, is like a **CCTV recording of one order processing scenario**, showing only the objects that become active during that moment — such as the Customer placing an order, the Order system processing each item, the shipping services dispatching the items, and the Notification service sending confirmation messages — and how these objects interact with each other step by step to complete that particular order.



Which classes or objects should appear in a UML sequence diagram?



**How many sequence diagrams
should typically be drawn for a
system?**

6.2 UML Sequence Diagram

Model the following scenario accurately with the use of UML Sequence Diagram.

(Hint: You may design your own scenario or select one from the following options.)

1. Enemy's turn: Goblin executes basic attack on Warrior
2. Player's turn: Warrior defends against Goblin's next attack
3. Enemy's turn: Goblin executes basic attack, causing lower damage to Warrior
4. Player's turn: Warrior executes Shield Bash on Goblin, stunning it
5. Enemy's turn: Goblin is stunned, skips turn
6. Player's turn: Warrior uses Potion to heal
7. Enemy's turn: Goblin is stunned, skips turn
8. Player's turn: Warrior executes basic attack, eliminating Goblin

Please ensure the characters are big enough, and the lines are not intersecting one another.



In practice, how do teams decide when to draw a new sequence diagram?

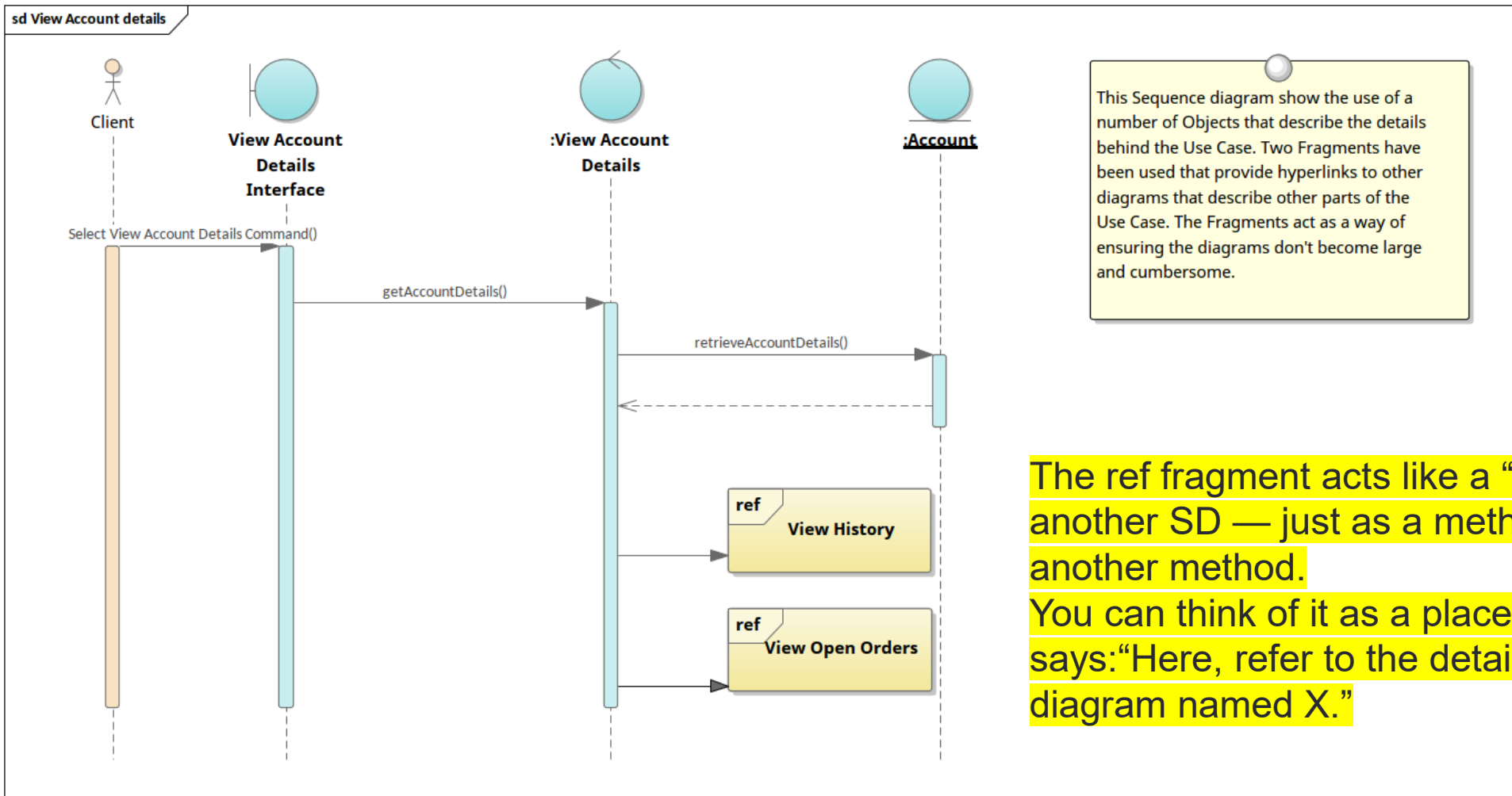
Key Takeaways

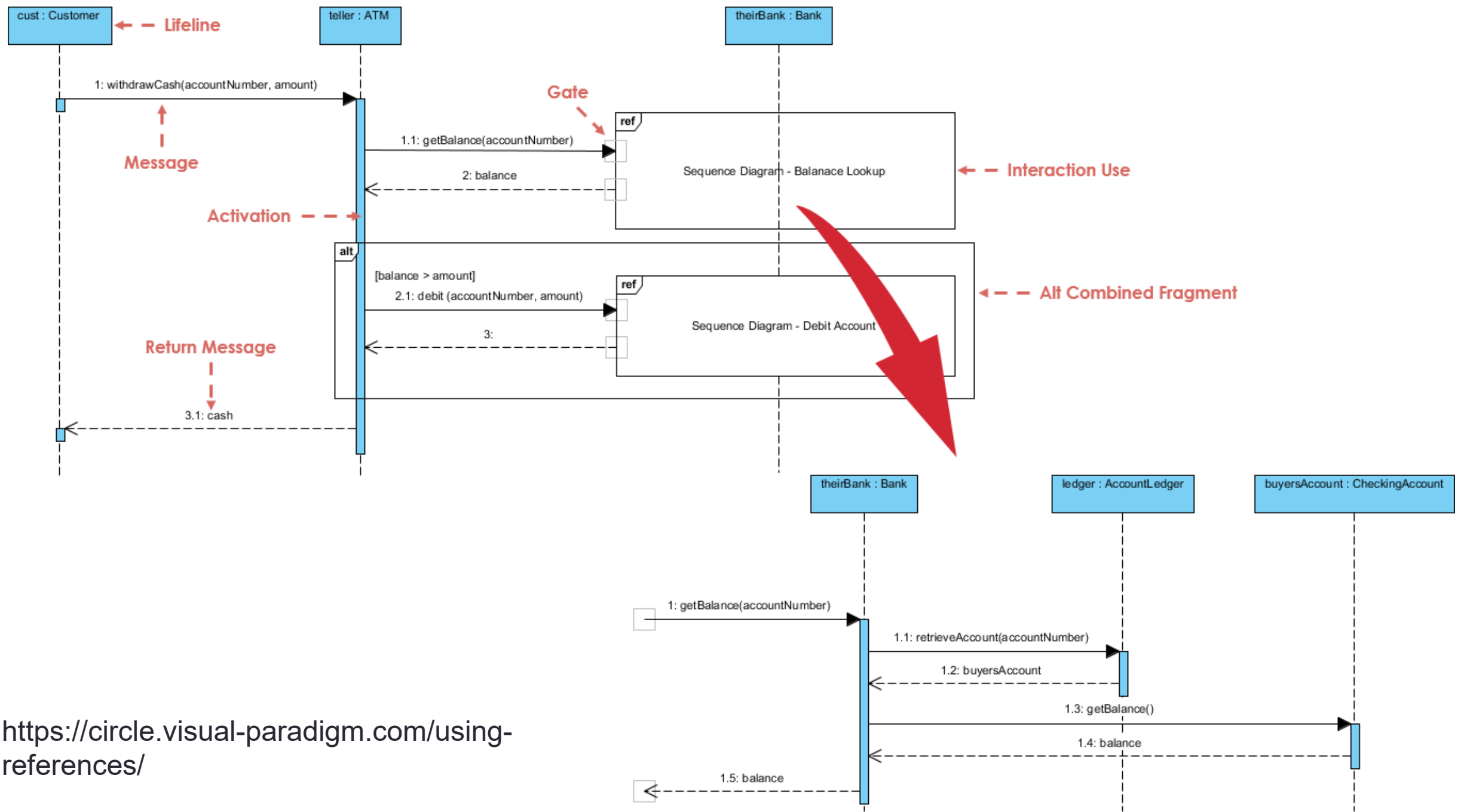
1. **There is no hard-and-fast rule** for the exact number of sequence diagrams.
2. **Create diagrams for each major use case, complex interaction, or critical scenario** that benefits from a time-ordered depiction of object interactions.
3. Avoid diagramming every trivial method call, as it can be time-consuming and provide little additional insight.

Option 1 — Multiple SDs

Option 2 — One Big SD

SD with references





<https://circle.visual-paradigm.com/using-references/>



What does a sequence diagram primarily depict?

UML Sequence Diagrams

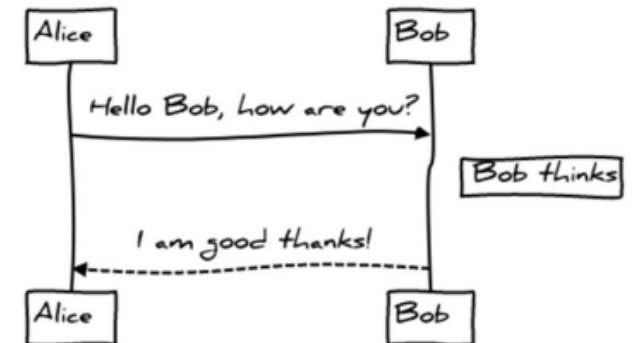
- Sequence diagrams, commonly used by developers, model the interactions between objects in a single use case. They illustrate how the different parts of a system interact with each other to carry out a function, and the order in which the interactions occur when a particular use case is executed.
- In simpler words, a sequence diagram shows **different parts of a system work in a 'sequence' to get something done.**

CX2006 Lecture note

Sequence Diagram

- Captures how a group of objects interact or collaborate to achieve a use case scenario.
- Usually captures the dynamics of a **single scenario** and the **sequence of messages** that are passed between the objects.

- Emphasizes the **time ordering** of messages.





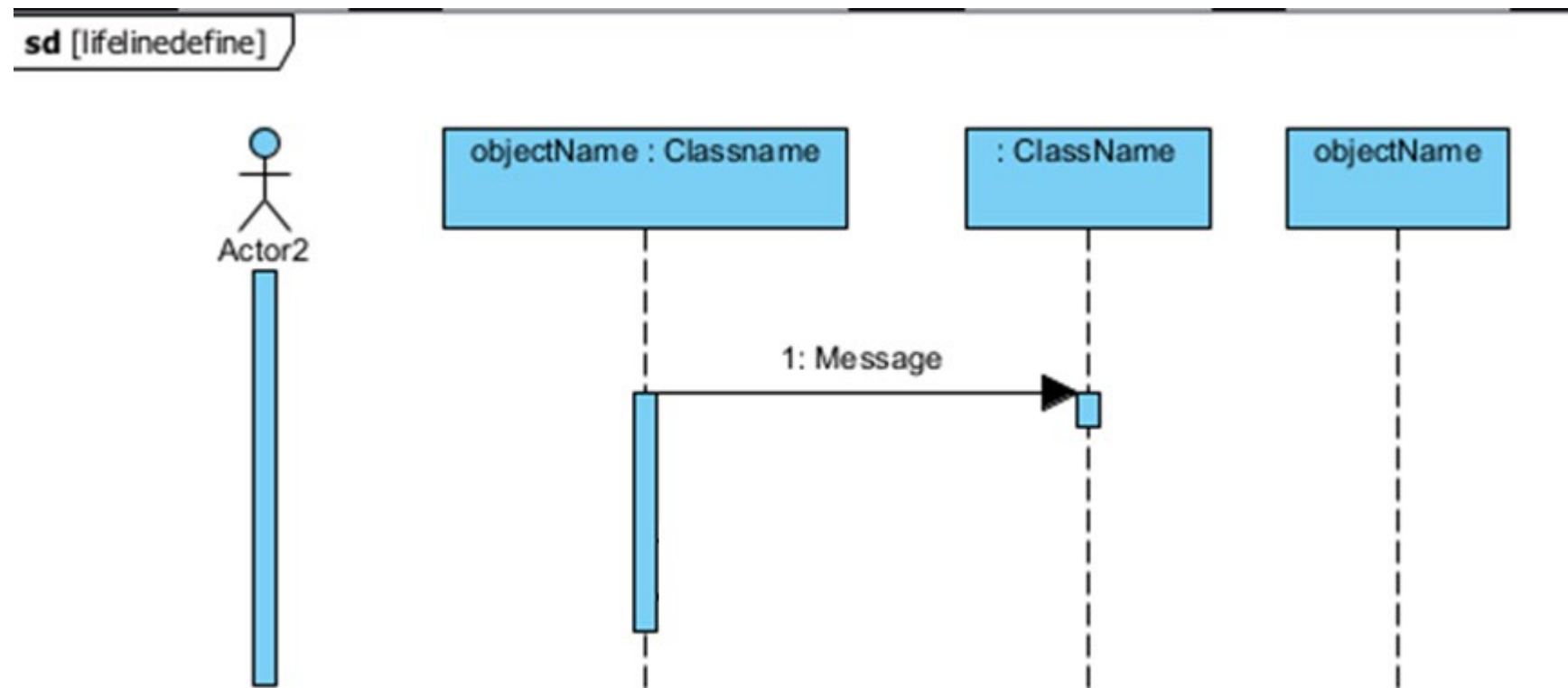
TOPIC

Topic 1: Sequence Diagram Elements

Sequence Diagram Elements

A Use Case = Actor + Goal + System Response

- Actor
- Object
- Lifeline
- Message
- Activation Bar





What does an actor represent in a UML sequence diagram?

Actors

An actor in a UML diagram represents a type of role where it interacts with the system and its objects. It is important to note here that an actor is always outside the scope of the system we aim to model using the UML diagram.



notation symbol for actor



Is an actor mandatory to be included in every UML sequence diagram?



1. The Dual Role of “**Member**” — Actor vs. Class

- In UML:
- **An Actor** represents a **role external to the system boundary** — someone or something that **interacts with** the system.
- **A Class** represents an **internal data structure or object** that the system uses **to manage or record** information.

When a member places an order in an online shopping system, the member is the **actor** interacting with the system from the outside. But inside the system, there is also a **Member** class in the database to store the member’s information, such as member ID, name, address, and contact details. The actor and the class refer to the same real-world person, but from two different perspectives — the actor is viewed from **outside the system boundary**, while the class is viewed from **inside the system** as part of the system’s internal data and design.

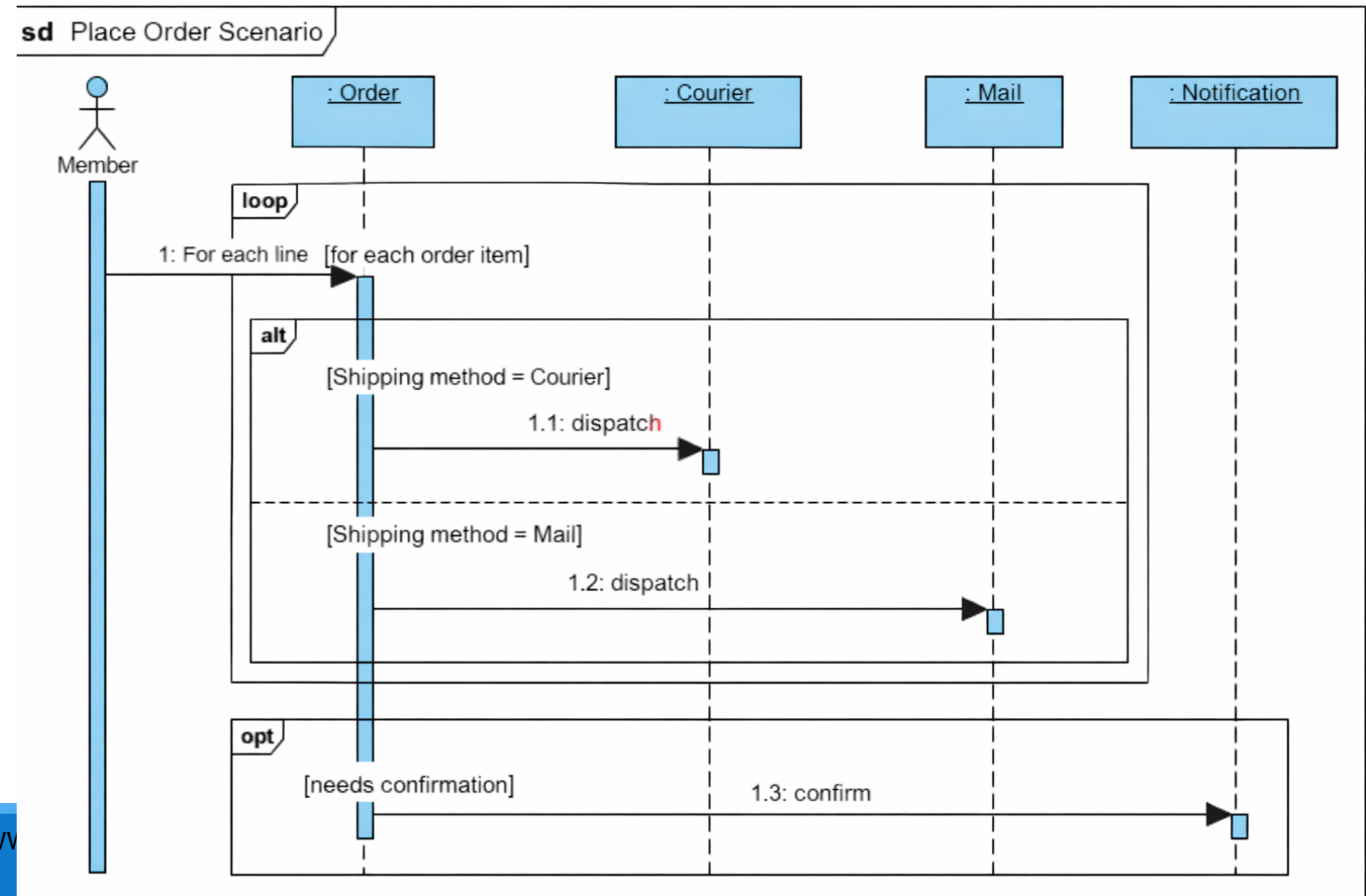
Online shopping Place order and delivery Key Characteristics

Element	Description
Actor	The external user or system that initiates the interaction.
System	The software or process being described.
Goal	The result or outcome the actor wants to achieve.
Main Success Scenario	The normal flow of events that lead to the goal.
Alternate / Exception Flows	Variations or error conditions in the process.

Sequence Diagrams show elements as they interact over time and they are organized according to object (horizontally) and time (vertically):

Object Dimension: The horizontal axis shows the elements that are involved in the interaction

Time
Dimension
The vertical
axis
represents
time
proceedings
(or
progressing)
down the
page.





What does an object in a sequence diagram represent?

Object

- Represents a class or object in UML. The object symbol demonstrates how an object will behave in the context of the system. Class attributes should not be listed in this shape.
- The standard in UML follows the following format – Instance Name : Class Name

objectName : Classname

: ClassName

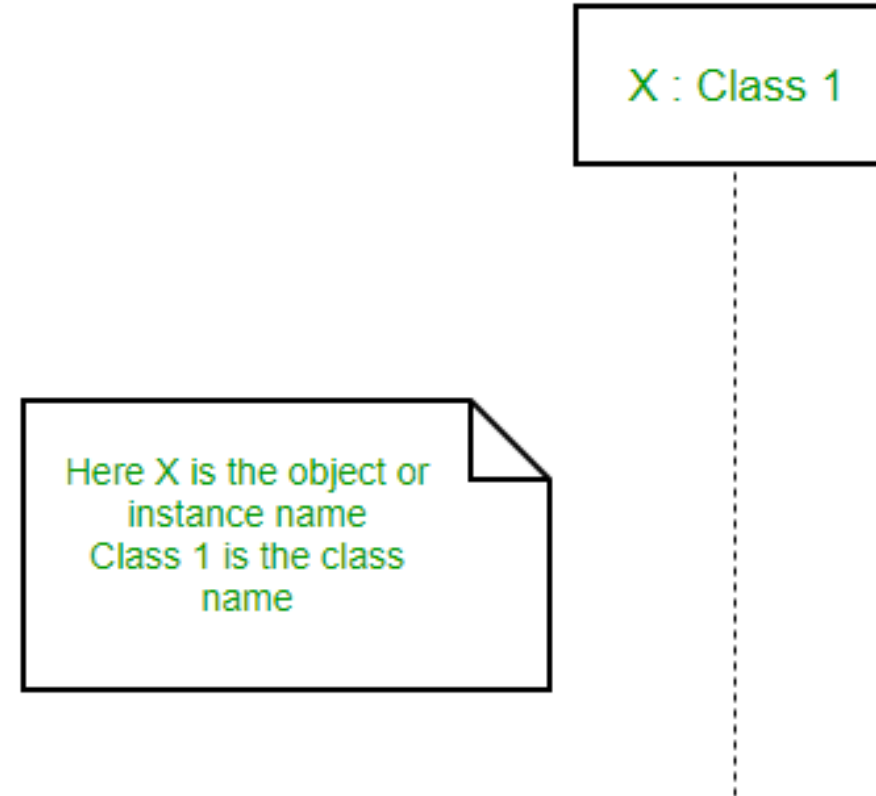
objectName



What does the lifeline associated with an object indicate?

Lifelines

- They represent the different objects or parts that interact with each other in the system during the sequence.
- A sequence diagram is made up of several of these lifeline notations that should be arranged horizontally across the top of the diagram.
- No two lifeline notations should overlap each other.





What do the horizontal arrows in a sequence diagram indicate?

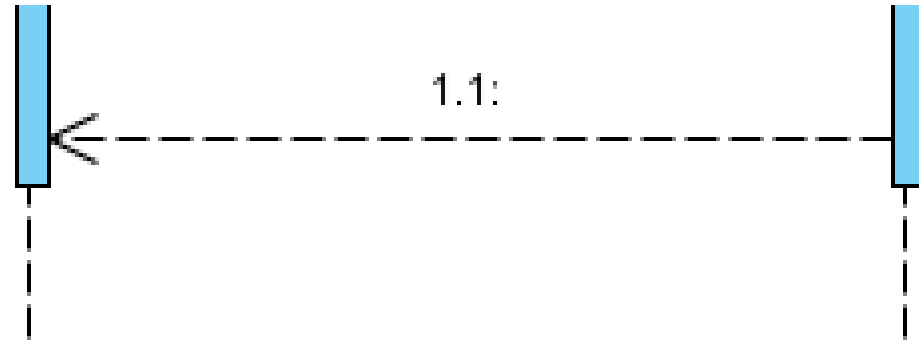
Call Message

- A message defines a particular communication between Lifelines of an Interaction.
- Call message is a kind of message that represents an invocation of operation of target lifeline.



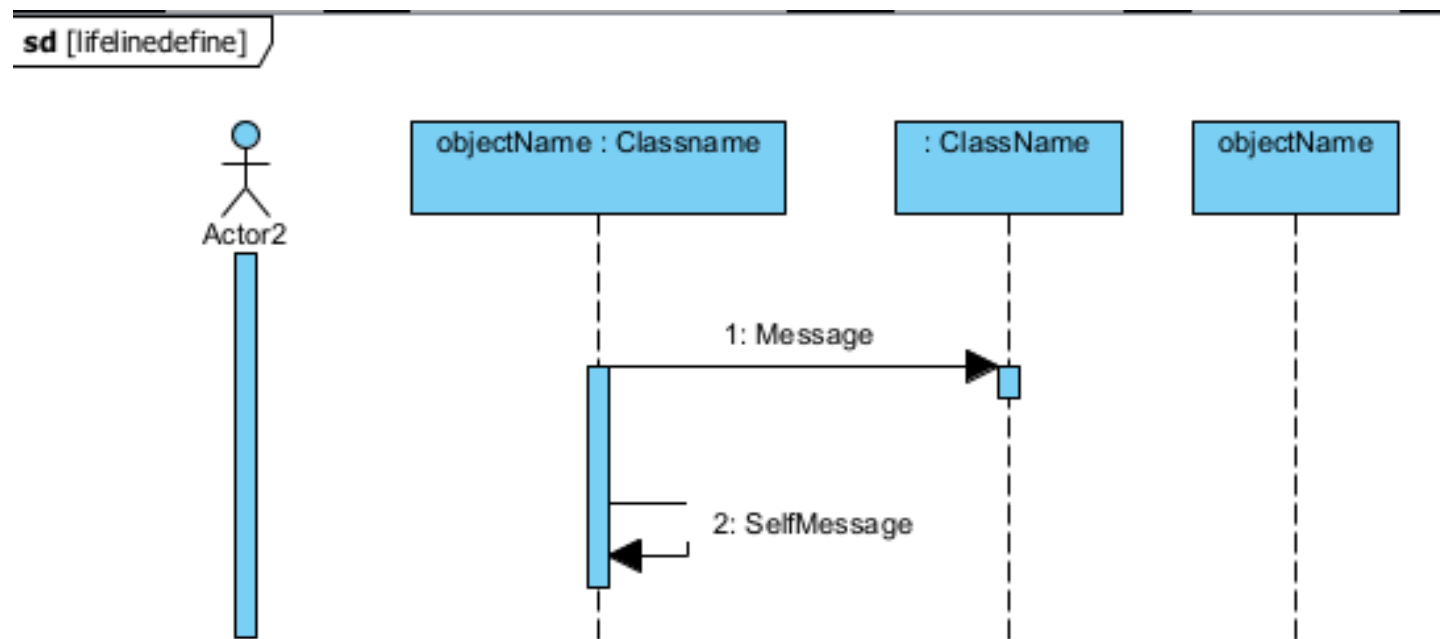
Return Message

- A message defines a particular communication between Lifelines of an Interaction.
- Return message is a kind of message that represents the pass of information back to the caller of a corresponded former message.



self message

- A self message is a message that an object sends to itself. It is a message that represents the invocation of message of the same lifeline. A self message can represent a recursive call of an operation, or one method calling another method belonging to the same object.



analyst : FinancialAnalyst

system : ReportingSystem

secSystem : SecuritySystem

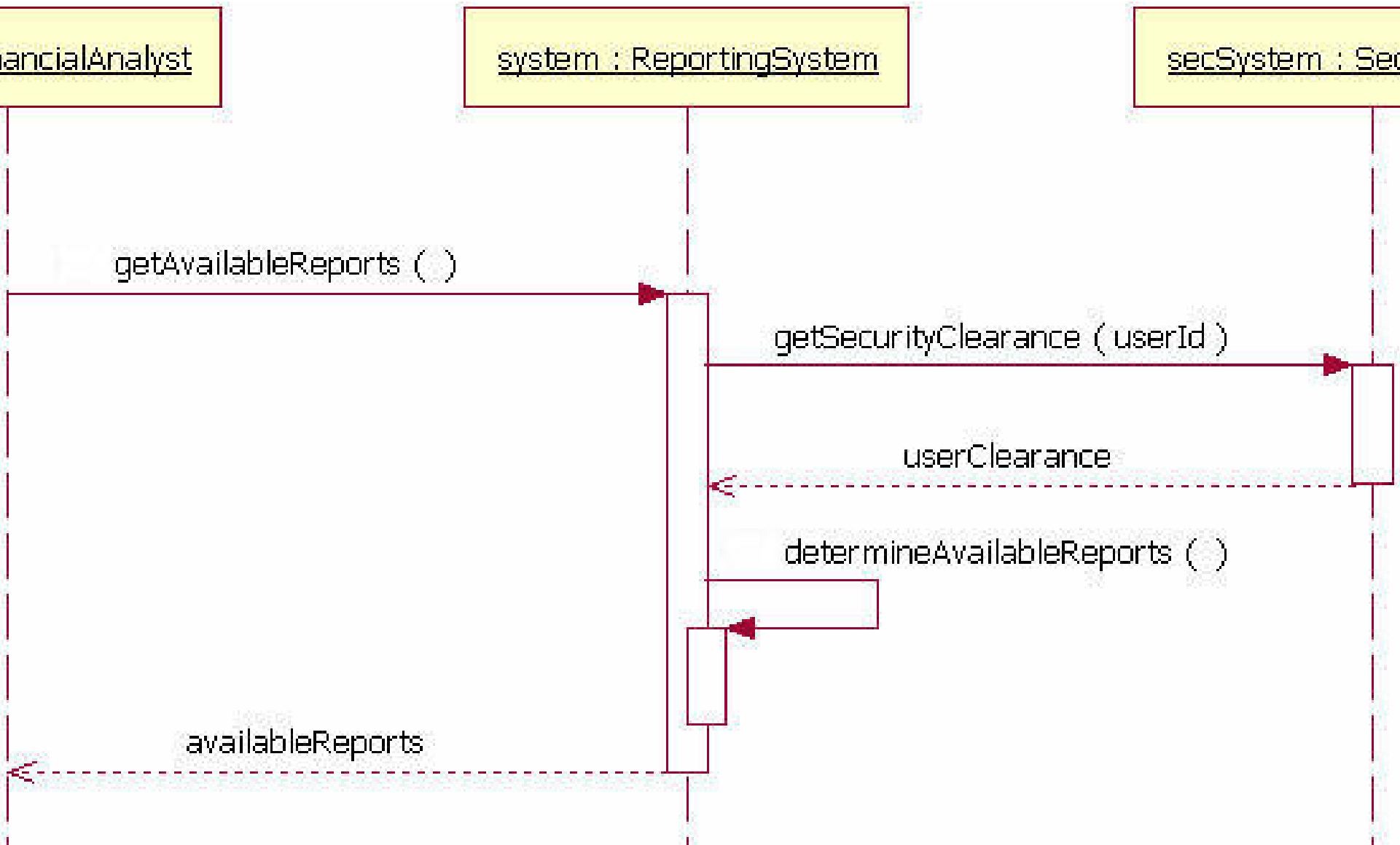
getAvailableReports ()

getSecurityClearance (userId)

userClearance

determineAvailableReports ()

availableReports

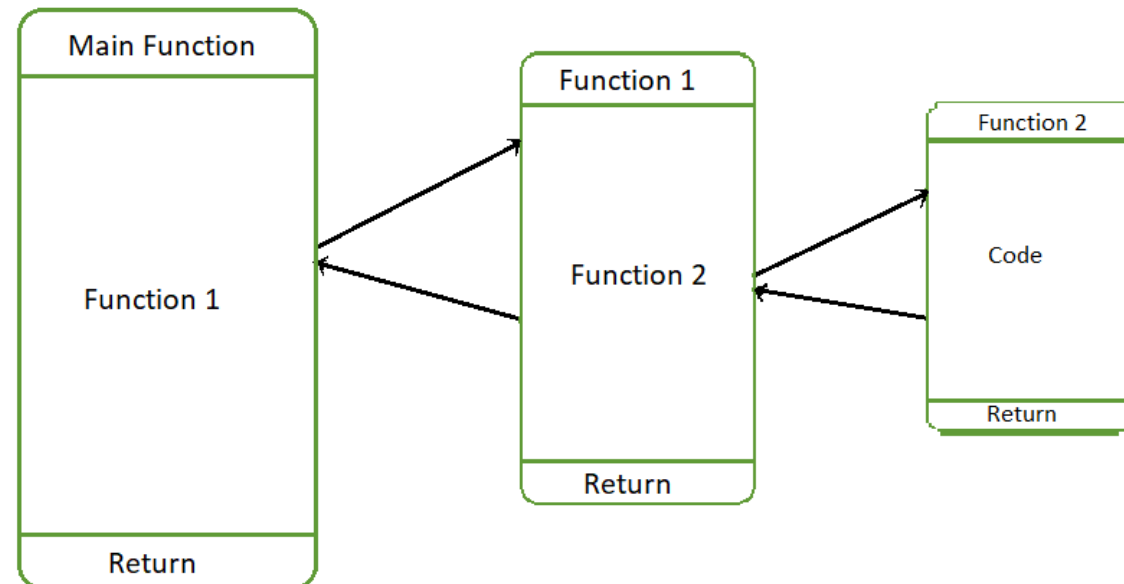
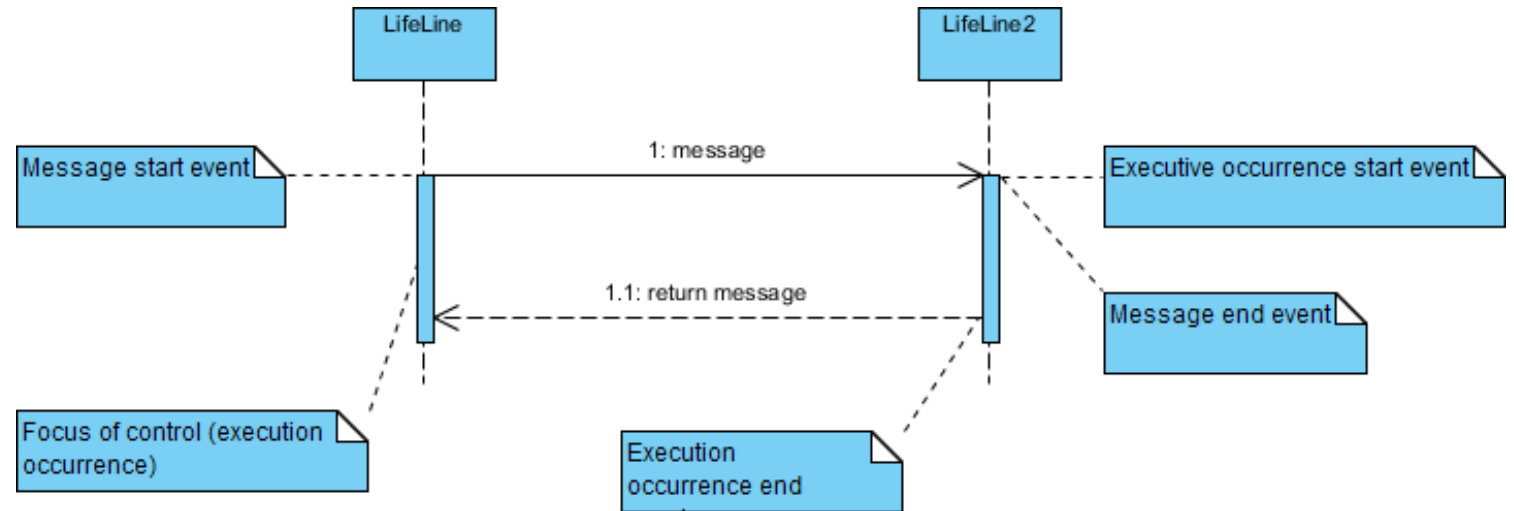




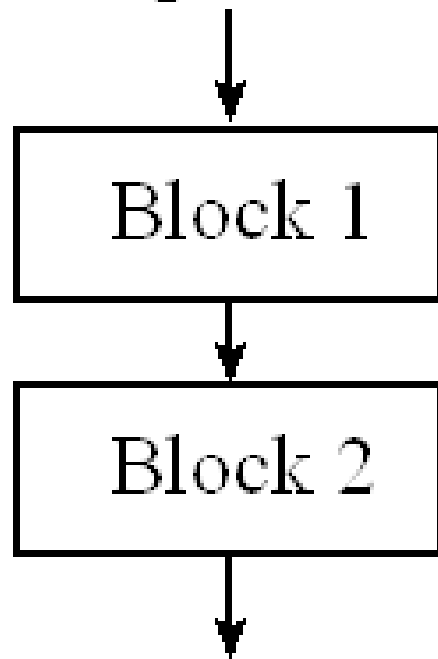
What does an activation bar (also known as the focus of control) illustrate in a sequence diagram?

Activation Bars

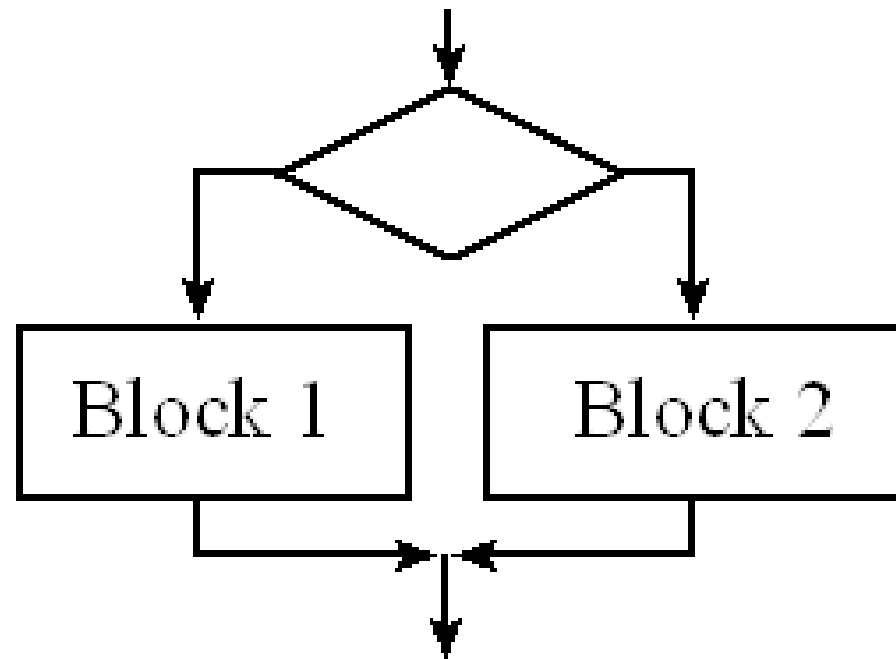
- Activation bar is the box placed on the lifeline. It is used to indicate that an object is active (or instantiated) during an interaction between two objects. The length of the rectangle indicates the duration of the objects staying active.
- In a sequence diagram, an interaction between two objects occurs when one object sends a message to another. The use of the activation bar on the lifelines of the Message Caller (the object that sends the message) and the Message Receiver (the object that receives the message) indicates that both are active/is instantiated during the exchange of the message.



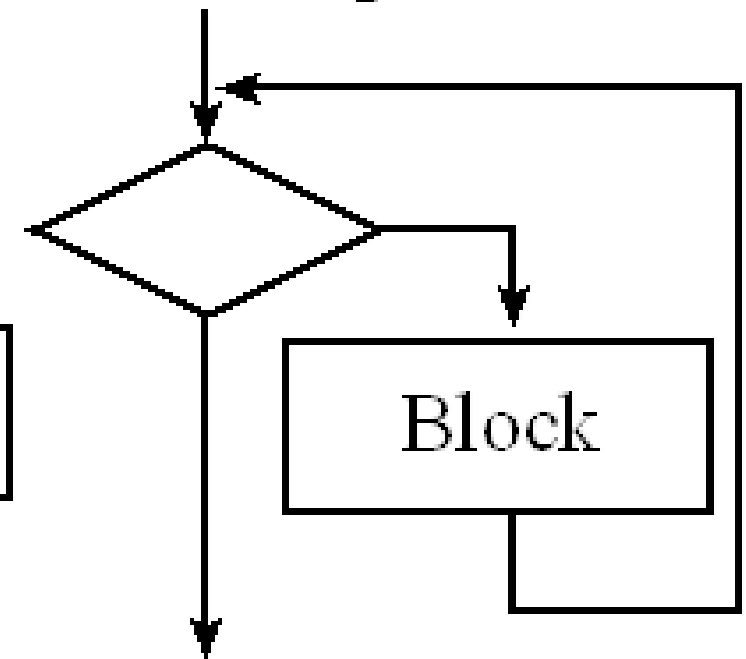
Sequence



Conditional

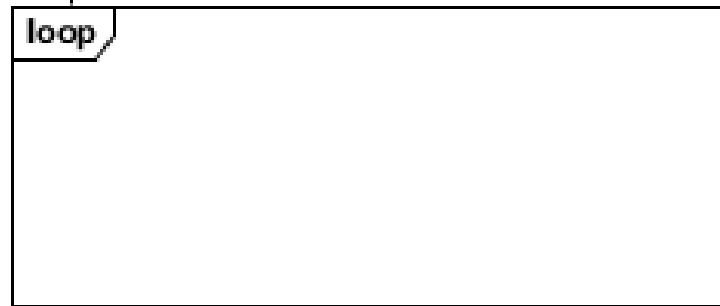


While-loop

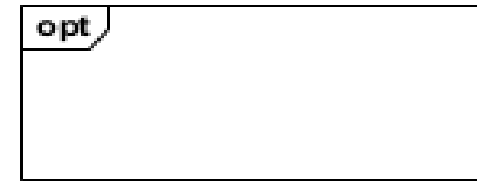


Combined Fragment

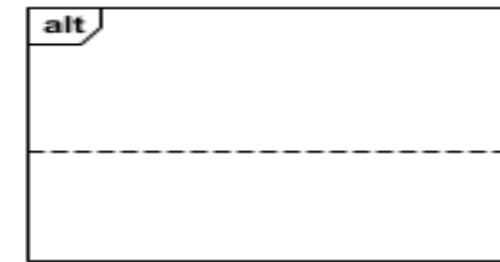
Fragment operator



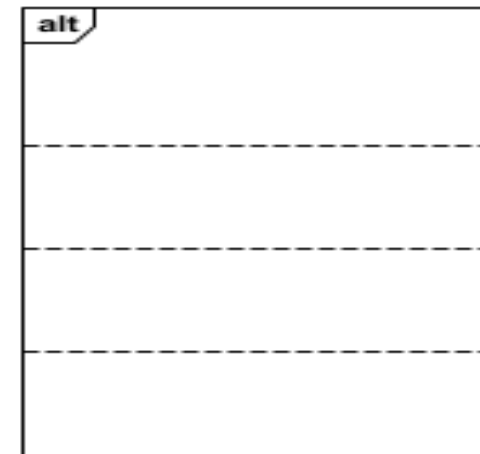
The sequence fragment box



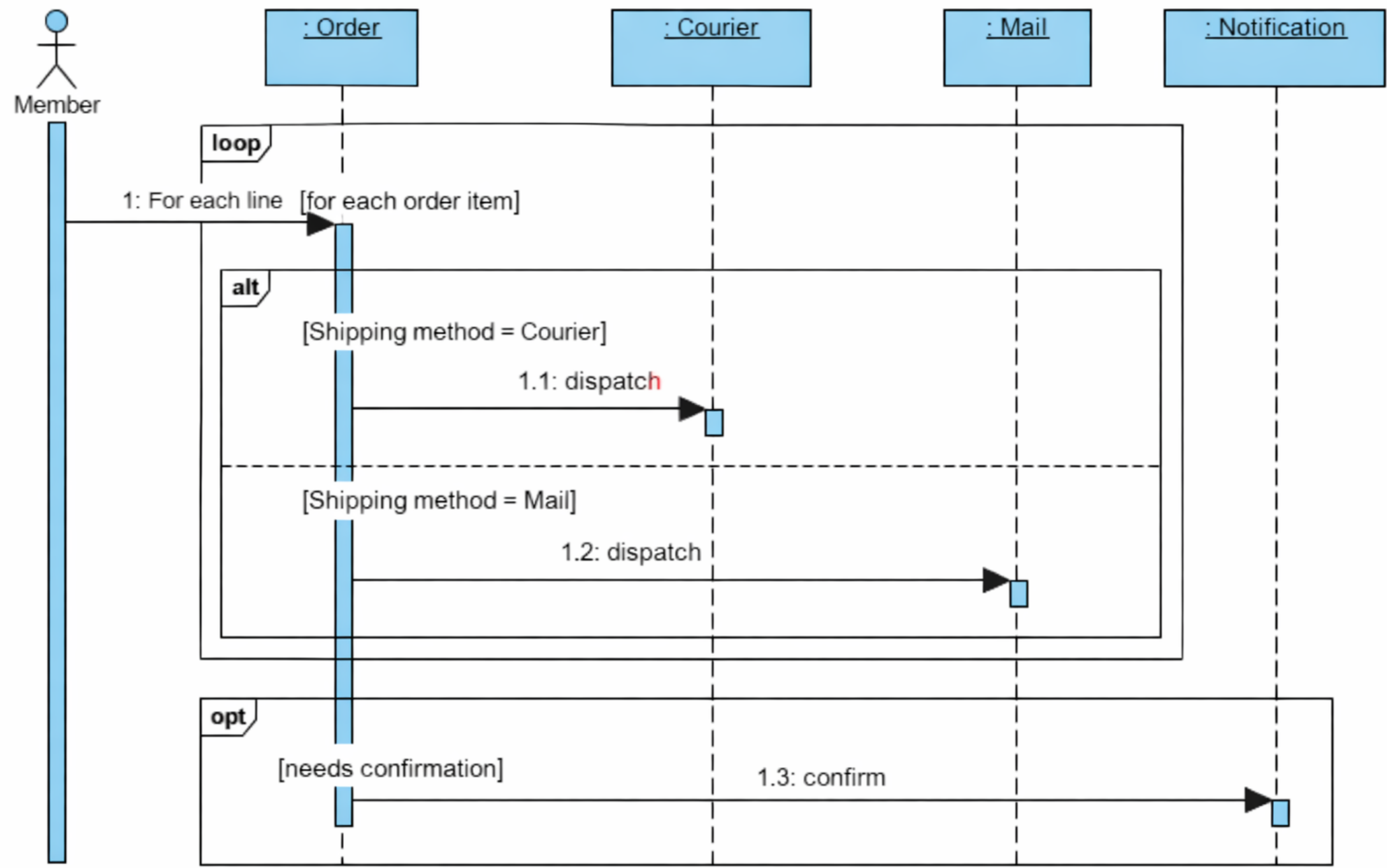
if



If...else
switch



sd Place Order Scenario



Project development process:3

3. Implementation

- **Coding Standards & Version Control:**
Follow best practices and use tools like Git.
- **Modular Development:**
Build self-contained classes and objects.
- **Testable Code:**
Design code that's easy to test and integrate.



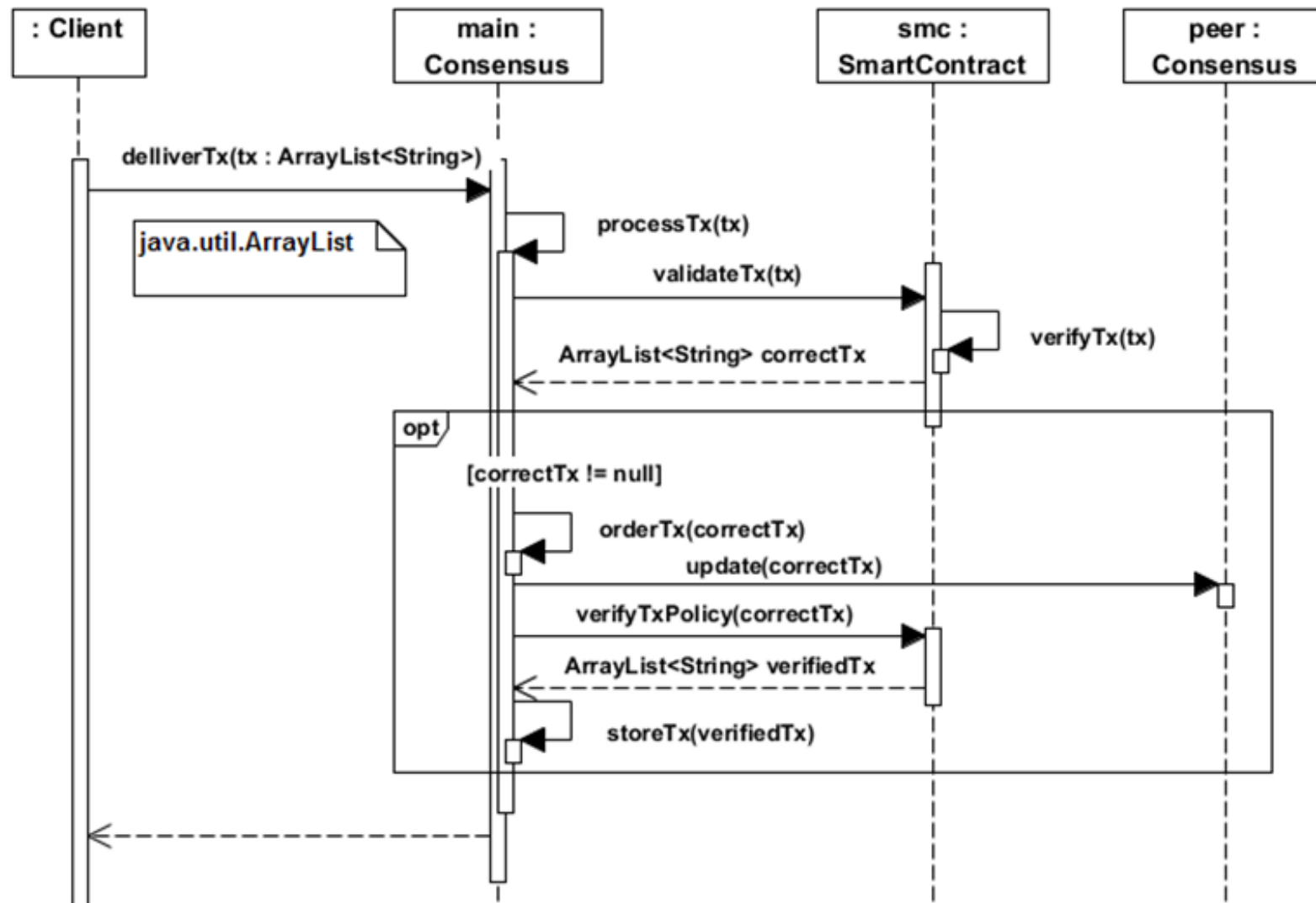
TOPIC

Topic 2: Diagram to Code

Past Year Paper 2017-2018S1 Q3b

- The UML Sequence Diagram in Appendix B (page 7) shows the objects' interaction of a scenario flow in a particular application. Using the details depicted in the diagram, write the preliminary JAVA code for the class Consensus and its methods. You may make appropriate assumptions on the method parameters, return types and return value(s) if they are not stated in the diagram.

(12 marks)



```

import java.util.ArrayList ; // 1

public class Consensus {
    private SmartContract smc ; // 1
    private Consensus peer; // 1

    public void deliverTx(ArrayList<String> tx) { // 1
        processTx(tx); // 1
    }

    private void processTx(ArrayList<String> tx) {
        ArrayList<String> correctTx = smc.validateTx(tx); // 1
        if (correctTx != null) { // 1
            orderTx(correctTx) ; // 1
            peer.update(correctTx); // 1
            ArrayList<String> verifiedTx = smc.verifyTxPolicy(correctTx); // 1
            storeTx(verifiedTx); // 1
        }
    }

    private void orderTx(ArrayList<String> tx) { // 1/2
    }

    private void storeTx(ArrayList<String> tx) { // 1/2
    }

    private void update(ArrayList<String> tx) { // 1
    }
}

```



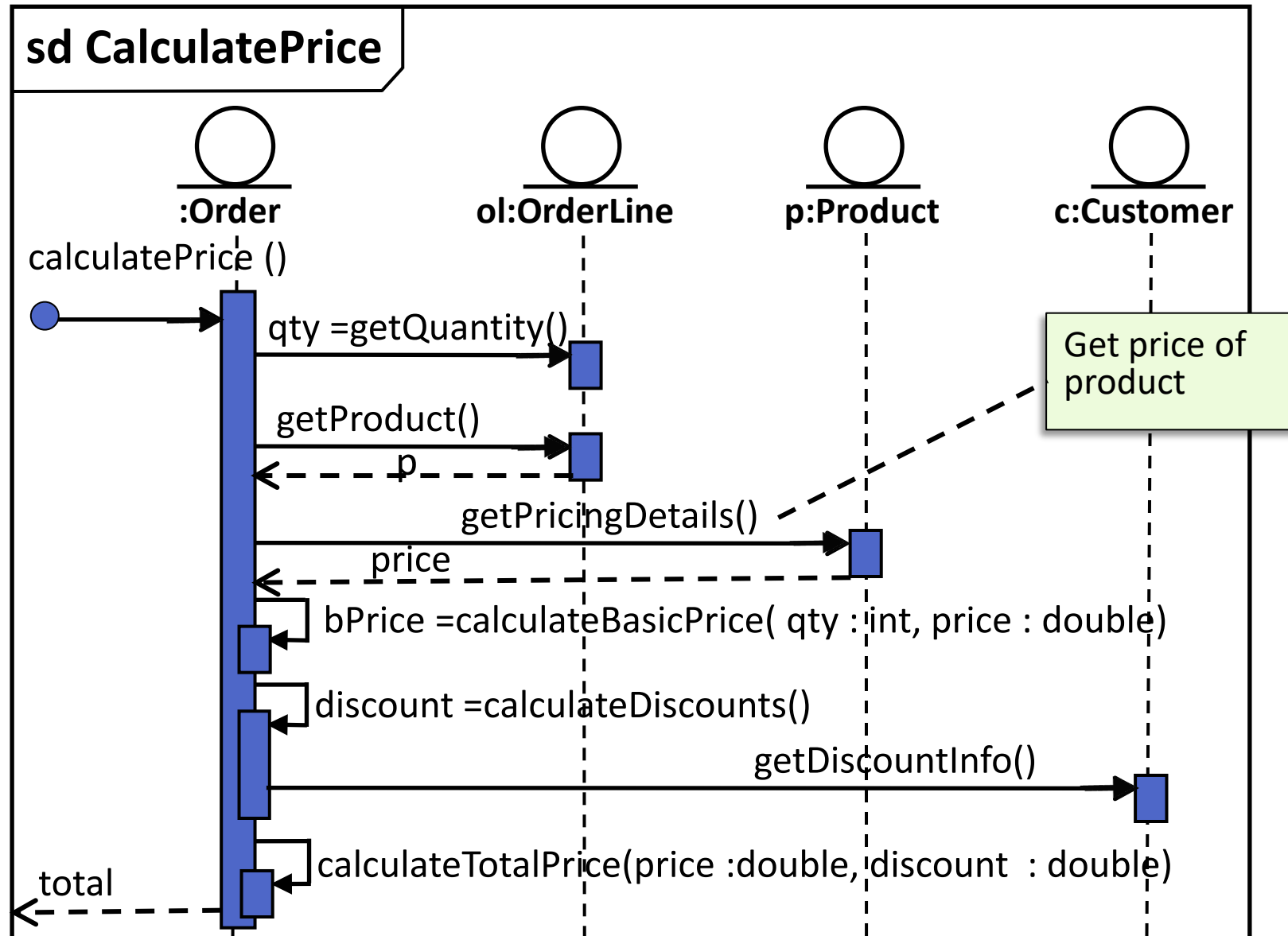
TOPIC

One more use case example

Calculate Price

- In this scenario, the system calculates the total price of an order. The order first obtains the quantity of the item from the order line and identifies the product associated with that order line. It then retrieves the price information of the product. Using the quantity and the product price, the order calculates the basic price of the items. Next, the system determines any applicable discounts by obtaining the customer's discount information. Finally, the order combines the basic price and the discount to compute the total price, and the resulting total amount is produced as the final price of the order.

Sequence Diagram – CalculatePrice





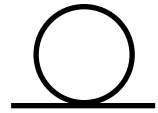
TOPIC

Class Diagram Topic 5: Class Stereotypes

BCE (Boundary–Control–Entity)

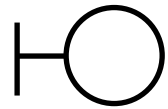
- **BCE is an object-oriented modeling pattern** proposed by *Ivar Jacobson* in the **OOSE (Object-Oriented Software Engineering)** method.
- To **structure early-stage system design** — before coding.
- It helps designers assign responsibilities to classes according to their roles in a system's use cases.

Class Stereotypes



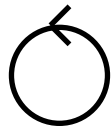
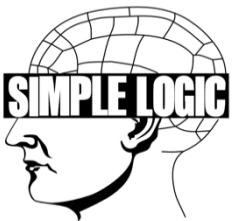
Entity

- Persistent **information** tracked by Program/Application/System



Boundary

- Interaction between System and External (System Surrounding) - **interfaces**

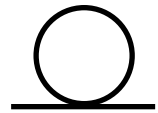


Control

- **Logic** to coordinate and realise use case (functional usage)

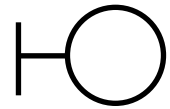


Class Stereotypes (Common Naming)



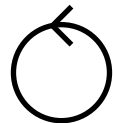
Entity

- Example: Student, Course, Group



Boundary

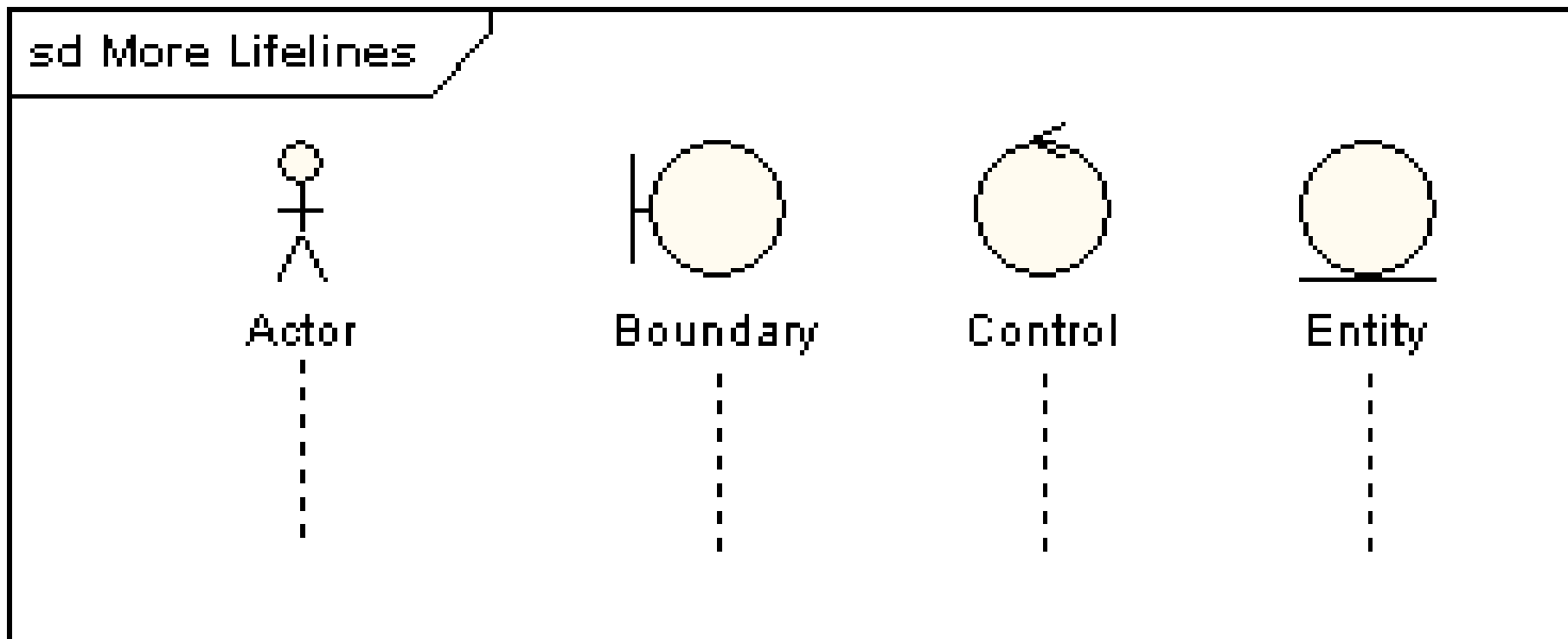
- Example: xxxUI, xxxForm, xxxInterface
- Example: InvoiceForm



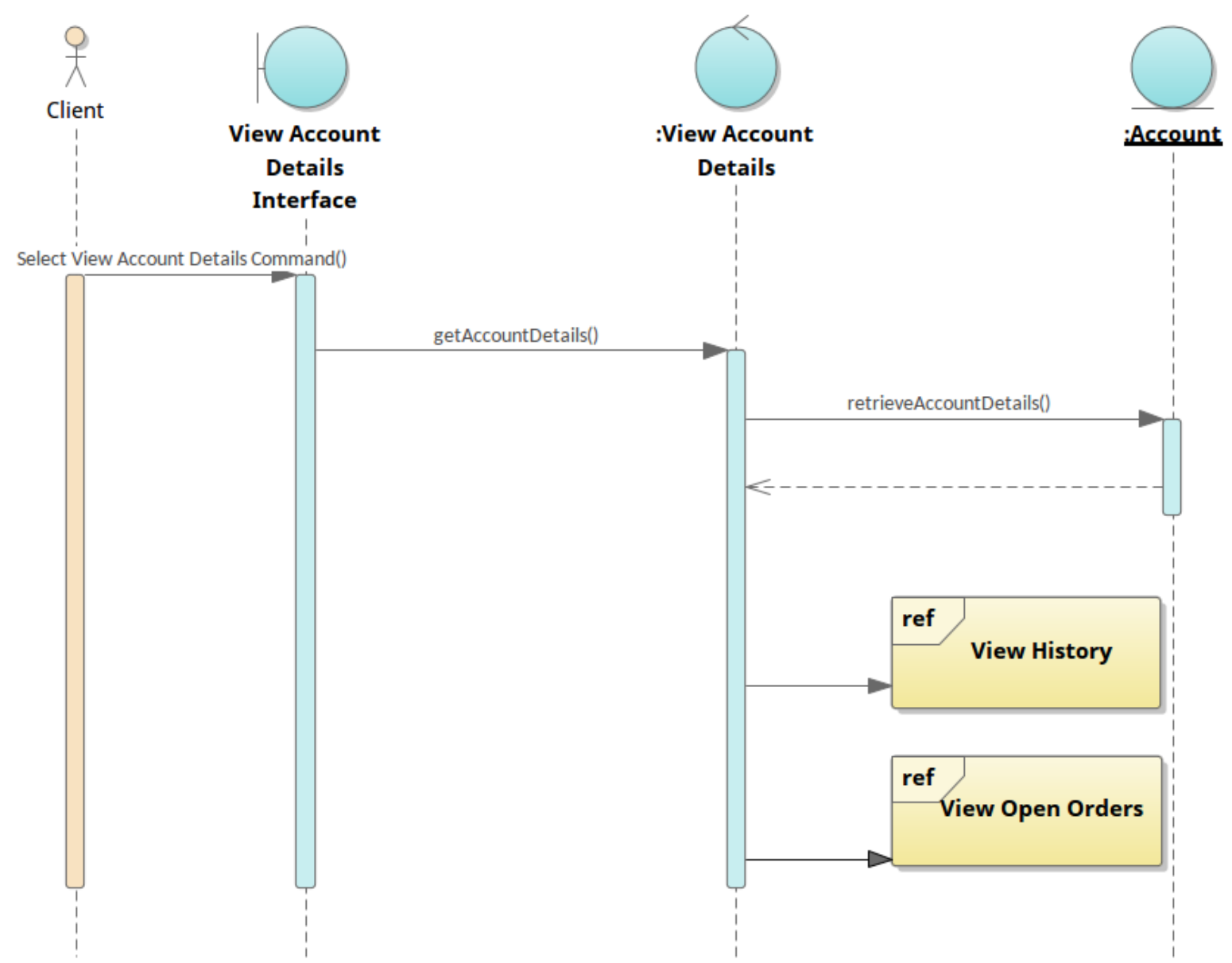
Control

- Example: xxxMgr, xxxCtrl, xxxController
- Example: CourseMgr

Entity, Control, or Boundary (ECB) Pattern



sd View Account details



This Sequence diagram show the use of a number of Objects that describe the details behind the Use Case. Two Fragments have been used that provide hyperlinks to other diagrams that describe other parts of the Use Case. The Fragments act as a way of ensuring the diagrams don't become large and cumbersome.